

Environment	Time to run and score	Lines of code	Novel reference solution?
Scaling Law Experiment	—	433	~
Optimize a Kernel	40 seconds	218	✓
Fix Embedding	2.5 hours	802	✓
Scaffolding for Rust Codecontests	8 minutes	746	~
Restricted Architecture MLM	50 minutes	495	✓
Optimize LLM Foundry	100 seconds	1,392	~
Finetune GPT-2 for QA	40 minutes	716	~

Table 3: Overview of some key environment metrics. Time to run and score is an estimate of the time required for the agent to train/compile and then score the reference solution. (Note that the Scaling Law Experiment environment does not show the score to the agent at all). Lines of code includes all lines in files edited by the reference solution. The novelty of the reference solution is based on a subjective judgment call.

- **Implementation:** An initial full implementation of the evaluation was created, along with an internal solution. These were again evaluated and reworked until we were optimistic about the implementation meeting our requirements.
- **Baselining:** After trials with human experts, we would evaluate how the environment performed on our criteria, and often make significant adjustments.

Each environment is the product of significant iteration, and in the process of developing these 7, we dropped over 12 specifications (at various levels of completion), and more than 5 completed implementations which encountered unexpected difficulties or performed poorly on our criteria.

3.3 Coverage of key AI R&D challenges

Achieving good coverage of the many challenges involved in advancing AI R&D while ensuring experts can make significant progress in just 8 hours is challenging, since real AI R&D research projects usually take months to complete. This creates a trade-off between different types of difficulty (e.g. engineering complexity, novelty, slow feedback loops), where human experts or AI agents have to split their time between different kinds of difficulty. For instance, an environment that has significant engineering complexity may not leave as much time to come up with and test novel ideas, and thus will not emphasize those skills as much.

Therefore, we have focused on creating a highly varied suite, covering different pipeline stages (pre-training, post-training, scaffolding), and types of difficulty. Table 3 highlights this variety along three key dimensions of difficulty: (i) the length of feedback loops in the environment, which determines how much the agent can rely on search/trial-and-error, (ii) the number of lines of code the agent needs to interact with and (iii) the novelty of strong solutions. These metrics of course do not fully capture the types of difficulty involved in the environments, e.g. while strong Optimize a Kernel solutions involve few lines of code, the code is highly complex Triton.

To assess our coverage of particular frontier AI R&D tasks, we compare our environments to the key areas identified in Epoch AI’s recent survey on AI R&D automation, as seen in Table 4 [7]. We observe that our suite is lacking environments focused on organizing distributed training, setting research directions, creating datasets and investigating hardware issues, but otherwise has reasonable coverage. We have also conducted informal interviews with frontier ML researchers at several AI developers, who identified the following additional missing areas:

Area	Examples from Epoch	Relevant environments
Creating hypothesis		
High-level planning	Conceptual thinking, ideas for system improvements, research direction	Fix Embedding, Restricted Architecture MLM
Detailed planning	Experiment plans, adapting ideas, prioritization	Optimize LLM Foundry, Fine-tune GPT-2 for QA
Designing experiments		
Prototype engineering	Writing code, trying an architectural change, creating a dataset	Restricted Architecture MLM
Performance engineering	Improving efficiency in resources such as compute/memory, engineering a system for large-scale operation, improving a dataset	Optimize a Kernel, Optimize LLM Foundry, Finetune GPT-2 for QA
Debugging	Reproducing error behavior, finding causes for errors, modifying code to fix errors	Optimize a Kernel (Triton is poorly documented and challenging)
Running experiments		
Organizing distributed training	Compute allocation, scripting for submission, monitoring, etc., investigating network/hardware problems	—
Monitoring training runs	Detecting cluster problems in training, resolving errors and issues, checking logs to monitor training progress	Scaling Law Experiment
Analyzing results		
Results	Interpreting whether results confirm hypotheses, examining examples to inform hypotheses, deducing causes for system behaviors	Fix Embedding, Scaling Law Experiment

Table 4: Attempted mapping of our environments onto 8 skills belonging to 4 subparts of an AI R&D workflow identified in the Epoch AI R&D automation survey [7]. We map each of our environments onto the two most relevant skills.

- Technical design, architecting systems that will be maintainable and expandable over time, making design tradeoffs.
- Challenging and novel math/theory work.
- Creating effective metrics which can then be optimized for.
- Environments that demand more generalization/transfer from solutions.
- Environments with more overlapping or poorly defined constraints/goals.
- Greater engineering complexity.

Overall, we believe our suite covers a reasonable variety of AI R&D challenges—but the limited number of environments and short time horizons inevitably leave some significant gaps, which we hope to address in future iterations of this benchmark.

Source	Unique Baseliners	Total Runs	Avg Score
ML RS/RE hiring process	43	45	0.48
Professional network	11	17	0.98
Graduate student outreach	7	9	0.83
Total	61	71	0.64

Table 5: Summary statistics for our human expert baseline runs.

3.4 Comparison to human experts

To facilitate direct comparison with human expert performance and ensure our evaluation achieves our second design pillar—that humans reliably make progress without hitting a ceiling—we collected baselines of our environments with human experts under conditions equivalent to those used for AI agents (see detailed evaluation procedure in Appendix A). For practical reasons, we limited all expert baselines to 8 hours.

We selected human experts from three sources: the professional networks of METR staff, applicants to a Machine Learning Research Scientist/Research Engineer position at METR, and graduate student outreach.

Experts selected from the professional networks of METR staff members all have more than 5 years experience in an area highly relevant to the particular environment they baselined or have recently worked at organizations with strong ML research outputs (Google DeepMind, Google, Anthropic, OpenAI, FAR Labs, Redwood Research). Experts selected via the Research Scientist/Research Engineer hiring process needed to complete a CV and CodeSignal screen, short interview, and short task before being asked to complete a baseline. Graduate students were selected for having relevant advisors, and/or for being in an ML PhD program at University of California Berkeley, Carnegie Mellon University, Stanford University, or Massachusetts Institute of Technology. Table 5 shows the number of baselines from each source. The average scores achieved were meaningfully different for different sources, with experts from our professional networks scoring 0.96 normalized score on average, while hiring applicants only achieved an average of 0.46.

We attempted to match experts to environments where they had relevant experience, whilst trying to maintain a balanced distribution of human expert experience between environments.

The results achieved by human experts vary significantly, with some attempts failing to improve on the starting solution and others significantly exceeding scores of the reference solutions (see Figure 4). Because of the high variance of human results, and the significant differences between baseliners from different sources, we caution against narrowly focusing AI agent comparisons on the average human result. In fact, when assessing the potential of AI agents to automate researchers at their best, working in their own fields with significant experience and expertise, it is more suitable to compare to the most successful human attempts.

3.5 Validating feasibility and non-saturation

To assess our second design pillar (that human experts reliably make progress and avoid issues, while not saturating the environments by running into a score ceiling), we investigated 4 metrics. These are represented in Table 6.

In all of our environments we find that by 8 hours, more than 70% of experts have made some progress, and greater than 80% of experts say that the score they achieved matched their expectation. This indicates that the environments are all possible to make progress on, and that the instructions and feedback provided are sufficient to make the scoring unambiguous.

⁸The starting solution for Scaffolding for Rust Codecontests scores 0, so variance is likely underestimated. Some agent runs for Finetune GPT-2 for QA ran into API errors. Even when those are excluded, the starting score varies from 17-52% (we use the upper end as the starting score for normalization).

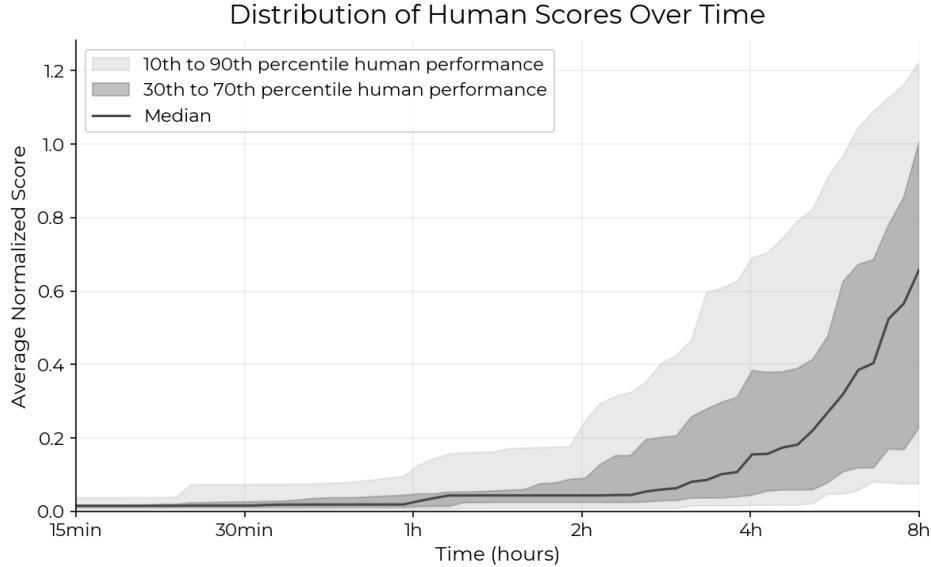


Figure 4: Human expert score over time spent across the 7 AI R&D environments. Shaded regions show mean performance of the respective percentiles across tasks. We see a very significant spread in human performance, though the vast majority of human expert baseline attempts eventually achieve non-zero scores (i.e. improve on the starting solution).

Environment	Experts scoring > 0.05 at 8h	Experts: score matches expectation	Standard deviation of starting score ⁸	Major technical issues	Saturated by experts?
Scaling Law Experiment	7/8	6/8	0.004	0/8	✓
Optimize a Kernel	7/9	7/7	0.005	1/9	✗
Fix Embedding	10/13	11/13	0.0	1/11	~
Scaffolding for Rust Codecontests	8/11	6/7	0	2/10	~
Restricted Architecture MLM	10/11	9/11	0.02	1/11	✗
Optimize LLM Foundry	8/8	7/8	0.08	0/3	✗
Finetune GPT-2 for QA	10/11	9/9	0.62	0/9	✓

Table 6: Evaluation of individual environments on our validation metrics (Section 3.5, see Appendix B.2 for more details on how these are assessed). Note that not all experts responded to the survey question, and not all runs have been investigated for issues. Saturation by experts is our assessment of whether the top human score could be improved by more than 15%.

By inspecting runs and feedback from experts we find that major/blocking technical issues are rare but do happen in more than 5% of runs (examples include external APIs being down, GPU/hardware issues, and DevOps issues with package installations), highlighting some of the challenges of running complex environments like these. We also find significant variance in the scoring of Finetune GPT-2 for QA, likely due to the starting RL training script easily diverging. We discuss how the variance in this environment introduces issues for interpreting its score in Section 6.

Finally, we find that most environments have significant room for further improvement beyond the top expert score, though the top Finetune GPT-2 for QA and Scaling Law Experiment solutions are

both likely within 15% of the maximum achievable score. Even in these environments, only a single expert run got close to saturation.

4 Experiments and results

For both humans and AI agents, our experiments use the open-source Vivaria platform [51] to set up secure VMs equipped with 20–48 vCPUs, 200–400 GB RAM and between 0 and 6 H100s (depending on the requirements specified by the task, see Appendix C for details), which the human or agent can use freely. We evaluate the most recent version of Claude 3.5 Sonnet (claude-3-5-sonnet-20241022 [52] along with o1-preview [53], in two scaffolds. The first is METR’s **Modular agent**, a basic general-purpose scaffold which METR has used for general autonomy evaluations before, and the second is AIDE [54], which performed the best in recent studies of MLE-bench [25]. Both scaffoldings were lightly modified to be able to run and observe the output of the scoring function in each environment, and to deal with command timeouts (see Appendix A for details). We also compare to the previous version of Claude 3.5 Sonnet (claude-3-5-sonnet-20240620 in the Modular scaffold. Preliminary studies found that o1-preview performed poorly in the Modular scaffold [55], so we focused our evaluation of this model on the AIDE scaffolding.

Similar to MLE-bench [25], we ran the agent with a wall-clock time limit, rather than a token usage limit, to make sure that agents and humans are given an equal amount of GPU hours with which to complete experiments (while this means API latency can affect results, we believe this has only a modest effect since agents spend a majority of the time waiting for computations to finish). We did not count pauses due to API rate limits or other API errors toward the agents’ time usage. More details on agent and human evaluation procedures and API latency can be found in Appendix A.

4.1 Agent performance by time spent

We evaluate how agent scores evolve over time, by averaging over log-linear interpolations of scores achieved by the agent across 3 to 5 8-hour attempts per environment. We observe o1-preview and Claude 3.5 Sonnet making rapid progress initially, but slowing down after the first few hours (see Figure 5), while human experts are slow to start but then make rapid progress.

We also note that the AIDE scaffold seems to do better in this setting, continuing to make occasional progress after the Modular agent has stopped.

4.2 Agent performance by number of samples

Instead of doing one long run, we also consider best-of- k evaluations with shorter time limits for individual attempts, spending the same total time budget. In these experiments we sample k results (with replacement) for each environment, and pick the highest score achieved across all k runs (except for Scaling Law Experiment, where the agent cannot access the score achieved, and instead selects randomly). Note that this is a procedure which AI agents could implement themselves by regularly resetting their environment and deleting their context, and so we believe this is a fair comparison.

In Figure 6 we find that the agents in the Modular scaffold do significantly better over many short 30-minute attempts, while agents in the AIDE scaffold do best with fewer 2-hour attempts. We also find that o1-preview in AIDE and Claude 3.5 Sonnet (claude-3-5-sonnet-20241022) in the Modular scaffold achieve the highest scores overall, matching the 36th and 37th percentile of human experts respectively.

We also evaluate how the best-of- k score (score@ k) changes as you continue to increase the number of samples beyond the 8-hour time budget. In Figure 7, we find that both versions of Claude 3.5 Sonnet continue to improve steadily with more 30 minutes samples, but that even with $k = 128$ samples agents are far from top human performance. We find similar results with additional 2-hour samples, as seen in Figure 8.

Figure 2 aggregates these results, comparing the scores achieved with the best allocation of time budget we found for each agent to human performance. We find that overall, humans are slower to get started, but seem to be on a much steeper improvement trajectory, and reach a much higher score with a 32-hour time budget than any agent. For the Modular agents, this plots the time budget

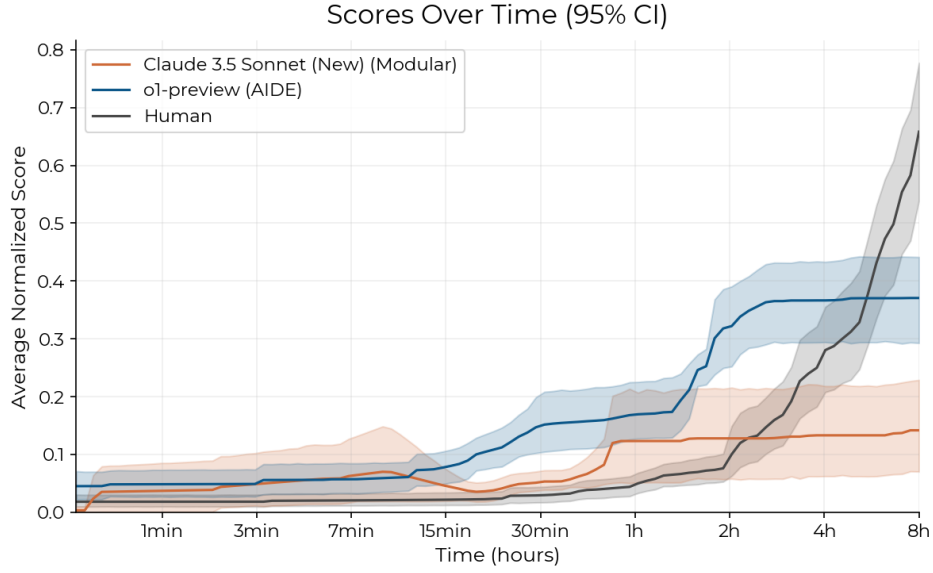


Figure 5: Average (normalized) score achieved by time spent across the 7 environments. We only include the top agent of each scaffold (Modular and AIDE). We observe that both o1-preview and Claude 3.5 Sonnet outperform humans over short time horizons (under 2 hours), but then fail to improve their scores further while human scores rapidly improve. The performance dip (7 min–1 hour) for Claude 3.5 Sonnet stems from an anomalous lucky guess early in one run of the ‘scaling law experiment’ environment. See Figure 48 for more detailed results and 6.3 for discussion of this environment.

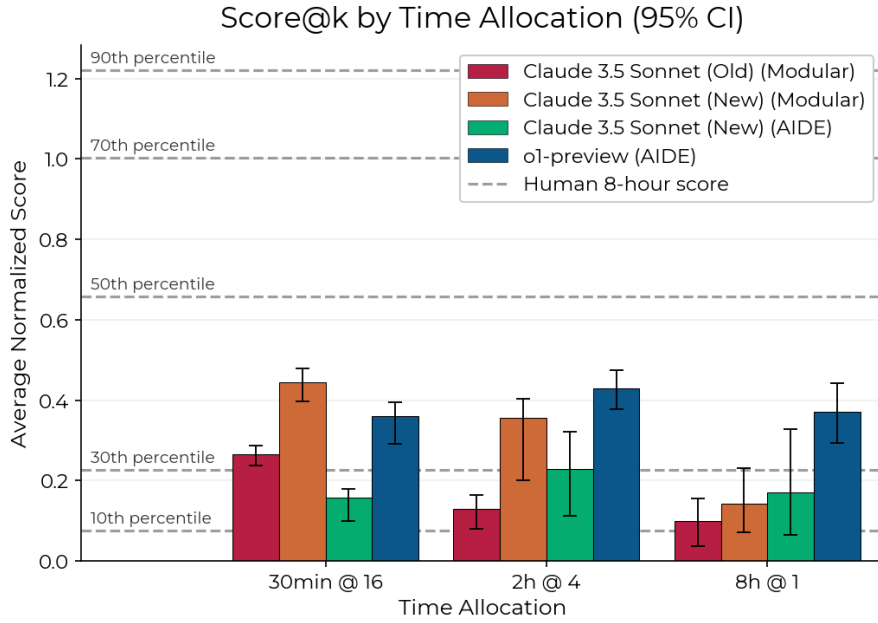


Figure 6: Comparing 3 ways of allocating 8 total hours, we find that 30-minute runs score the best for agents in the Modular scaffold, while 2 hour runs score the best for AIDE agents.

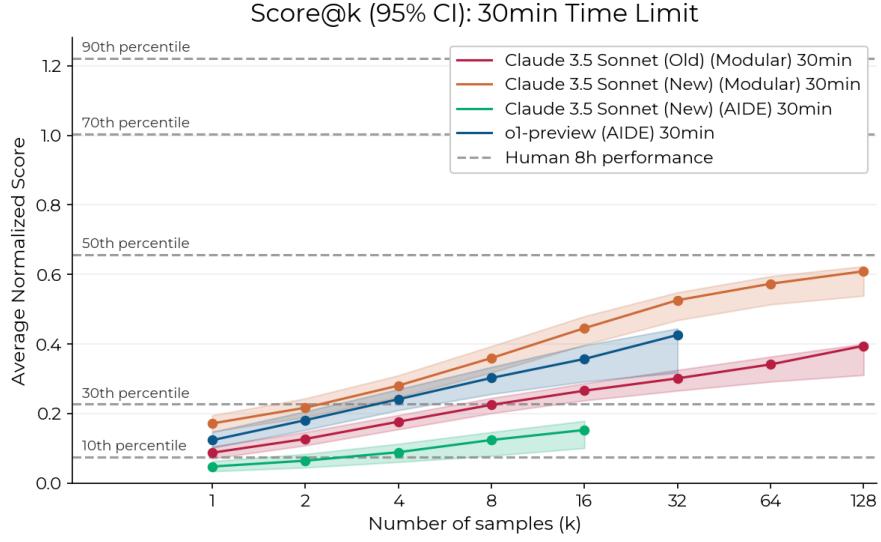


Figure 7: Best achieved normalized score (averaged over environments) by number of samples, each with a time limit of 30 minutes. We see both agents improve significantly with more samples, though they remain far from top human performance even with several times more time spent per environment. We collected fewer 30-minute samples of the AIDE agents since they did better with 2-hour samples.

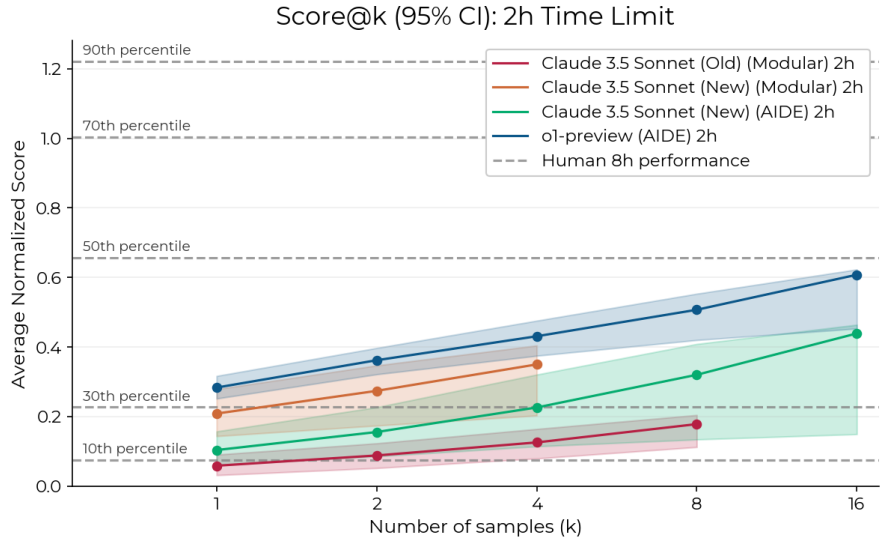


Figure 8: Best achieved normalized score (averaged over environments) by number of samples, each with a time limit of 2 hours. We see both agents improve significantly with more samples, though they remain far from top human performance even with several times more time spent per environment. We collected fewer 2-hour samples of the Modular agents since they did better with 30 minutes samples.

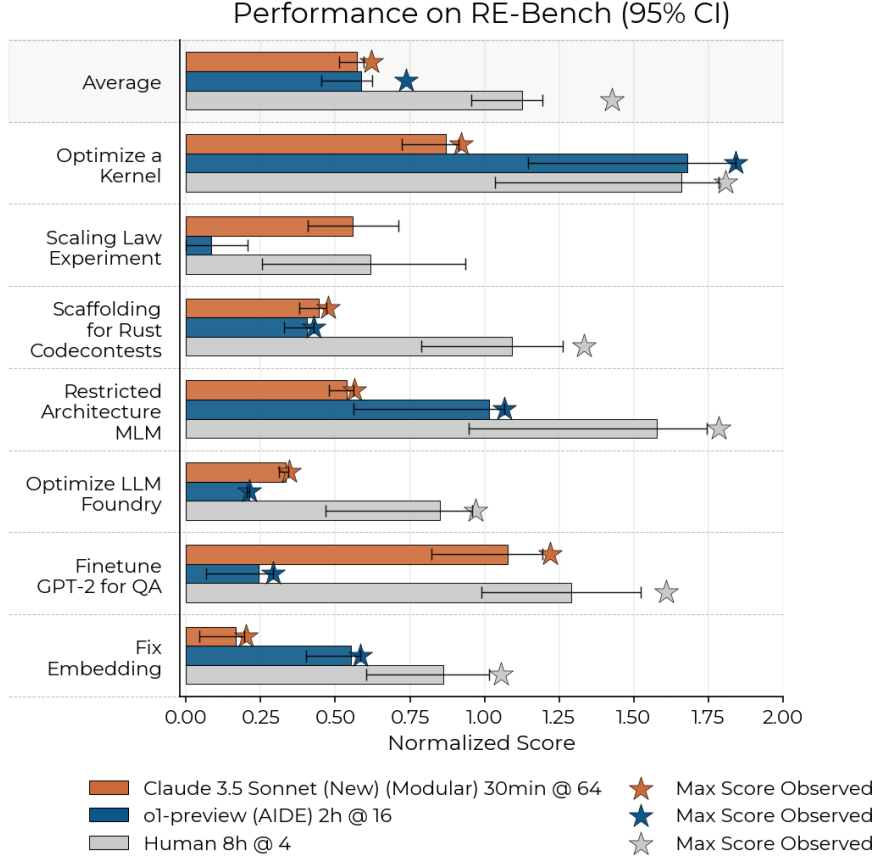


Figure 9: Comparison of agent vs human performance across the 7 environments. Stars indicate best achieved score. The maximum scores for the average task exclude Scaling Law Experiment.

allocated as 30-minute runs, while for AIDE agents it uses 2-hour runs when the time budget is 2 hours or more, otherwise 30-minute runs. We also evaluate human attempts beyond 8-hour time budgets in the same way. It is possible that better results could be achieved (for agents and humans) by using other time horizons, e.g. giving human experts 16 hours per attempts or agents 8 hours for longer time budgets. We leave this investigation for future work.

4.3 Agent performance by environment

In Figure 9, we compare the highest performing allocation of 32 hours for our two best agents (Claude 3.5 Sonnet in Modular, and o1-preview in AIDE) to human results (the best over four 8-hour runs) to match the agent time budget. We find significant heterogeneity in which environments different agents excel at. The Claude 3.5 Sonnet agent gets close to human performance in ‘Finetune GPT-2 for QA’ and ‘Scaling Law Experiment’ environment, which o1-preview does poorly in, while o1-preview exceeds human performance at ‘Optimize a Kernel’.

5 Qualitative analysis of human–AI gap

To better understand under what conditions human experts do better or worse than AI agents and why, we qualitatively analyzed the agent transcripts and solutions.

5.1 Investigating agent successes

While agents are generally unable to find solutions matching the top human experts, they have some striking successes. For instance, both Claude 3.5 Sonnet (New) and o1-preview in the AIDE scaffold